Błażej Pawluk, 279738

WSI, Laboratorium, Lista 1

Pobranie danych ze zbioru EMNIST MNIST

```
from tensorflow.keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.astype('float32') / 255.0
x_train = x_train.reshape(x_train.shape[0], -1)

x_test = x_test.astype('float32') / 255.0
x_test = x_test.reshape(x_test.shape[0], -1)

y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)
```

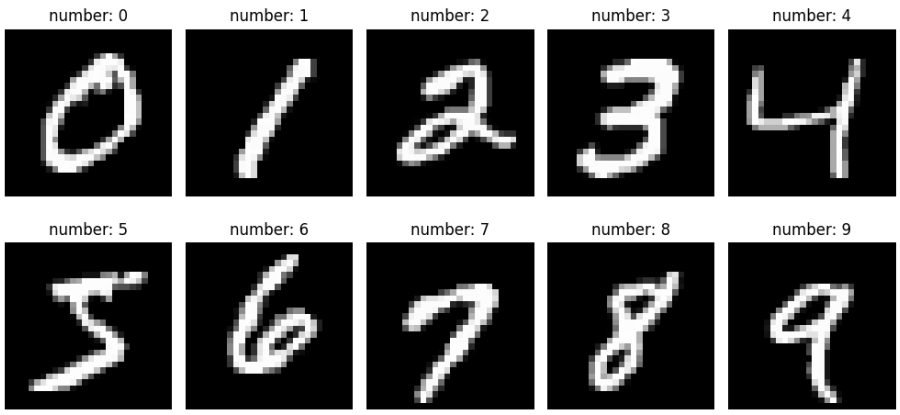
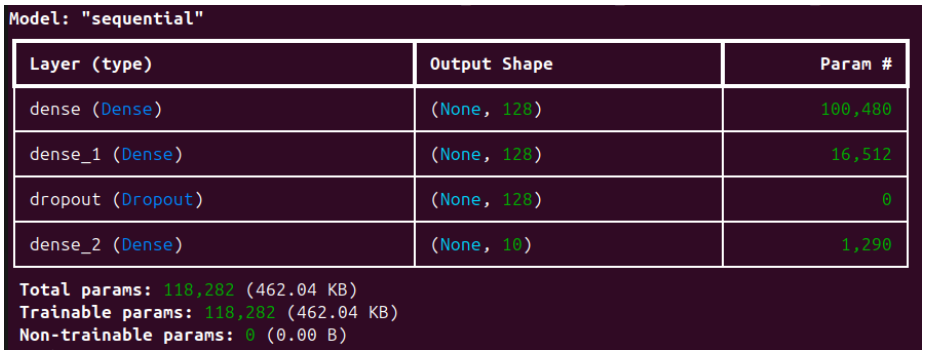


Figure 1: Cyfry ze zbioru EMNIST MNIST, na których sieć jest trenowana

Tworzenie i kompilacja modelu

```
# tworzenie modelu
model = Sequential()
model.add(Dense(units=128, input_shape=(784,),
    activation='relu'))
model.add(Dense(units=128, activation='relu'))
model.add(Dropout(0.25))
model.add(Dense(units=num_classes, activation='softmax'))

# kompilacja modelu
model.compile(
        loss='categorical_crossentropy',
        optimizer='adam',
        metrics=['acc', 'precision', 'recall']
)
model.summary()
```



Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 128)	16,512
dropout (Dropout)	(None, 128)	0
dense_2 (Dense)	(None, 10)	1,290

Total params: 118,282 (462.04 KB)
Trainable params: 118,282 (462.04 KB)
Non-trainable params: 0 (0.00 B)

Figure 2: Schemat sieci, pokazany dzięki model.summary()

Trenowanie modelu i statystyki z treningu

```
# trenowanie modelu
train = model.fit(
        x=x_train,
        y=y_train,
        batch_size=512,
        epochs=10
)

# wyświetlenie statystyk treningu
acc = train.history['acc']
```

```
loss = train.history['loss']
precision = train.history['precision']
recall = train.history['recall']

fig, ax1 = plt.subplots()
ax2 = ax1.twinx()

ax1.set_xlabel('epoka')
ax1.set_ylabel('accuracy / precision / recall')
ax2.set_ylabel('loss')

ax1.plot(range(1, len(acc) + 1), acc, label='accuracy')
ax1.plot(range(1, len(precision) + 1), precision,
    label='precision')
ax1.plot(range(1, len(recall) + 1), recall,
    label='recall')
ax2.plot(range(1, len(loss) + 1), loss, label='loss')

fig.legend()
plt.title('training statistics')
plt.tight_layout()
plt.show()
```

```
Epoch 1/10
118/118 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step - acc: 0.6834 - loss: 1.0354 - precision: 0.8858 - recall: 0.4792
Epoch 2/10
118/118 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - acc: 0.9298 - loss: 0.2422 - precision: 0.9474 - recall: 0.9110
Epoch 3/10
118/118 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - acc: 0.9502 - loss: 0.1728 - precision: 0.9631 - recall: 0.9389
Epoch 4/10
118/118 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - acc: 0.9604 - loss: 0.1288 - precision: 0.9685 - recall: 0.9537
Epoch 5/10
118/118 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - acc: 0.9665 - loss: 0.1100 - precision: 0.9735 - recall: 0.9611
Epoch 6/10
118/118 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - acc: 0.9732 - loss: 0.0897 - precision: 0.9782 - recall: 0.9684
Epoch 7/10
118/118 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - acc: 0.9772 - loss: 0.0757 - precision: 0.9818 - recall: 0.9730
Epoch 8/10
118/118 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - acc: 0.9790 - loss: 0.0677 - precision: 0.9835 - recall: 0.9757
Epoch 9/10
118/118 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - acc: 0.9823 - loss: 0.0570 - precision: 0.9850 - recall: 0.9793
Epoch 10/10
118/118 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - acc: 0.9825 - loss: 0.0567 - precision: 0.9855 - recall: 0.9796
```

Figure 3: Kolejne epoki i ich statystyki

Testy na zbiorze MNIST

```
# testowanie na zbiorze MNIST
print("testing on MNIST dataset:")
loss, acc, precision, recall = model.evaluate(x_test,
    y_test)
```

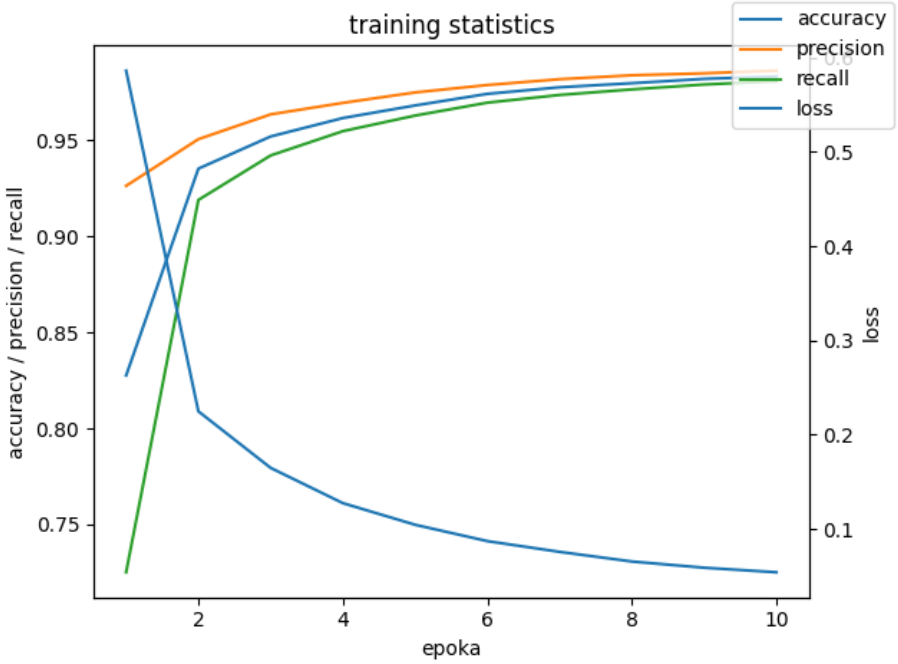


Figure 4: Statystyki: accuracy, precision, recall, loss, w treningu sieci neuronowej

```
print(f"accuracy: {acc*100:.2f}%")
print(f"loss: {loss*100:.2f}%")
print(f"precision: {precision*100:.2f}%")
print(f"recall: {recall*100:.2f}%")

y_pred_prob = model.predict(x_test)
y_pred = np.argmax(y_pred_prob, axis=1)
y_real = np.argmax(y_test, axis=1)
conf_matrix = confusion_matrix(y_real, y_pred)

fig, ax = plt.subplots(figsize=(15, 10))
ax = sb.heatmap(conf_matrix, annot=True, fmt='d', ax=ax)
ax.set_xlabel('prediction')
ax.set_ylabel('real')
ax.set_title('confusion matrix (testing on MNIST
    dataset)')

plt.tight_layout()
```

```
plt.show()
```

```
testing on MNIST dataset:
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - acc: 0.9737 - loss: 0.0868 - precision: 0.9764 - recall: 0.9709
accuracy: 97.72%
loss: 7.45%
precision: 97.99%
recall: 97.46%
```

Figure 5: Statystyki: accuracy, precision, recall, loss, w teście na zbiorze EMNIST MNIST

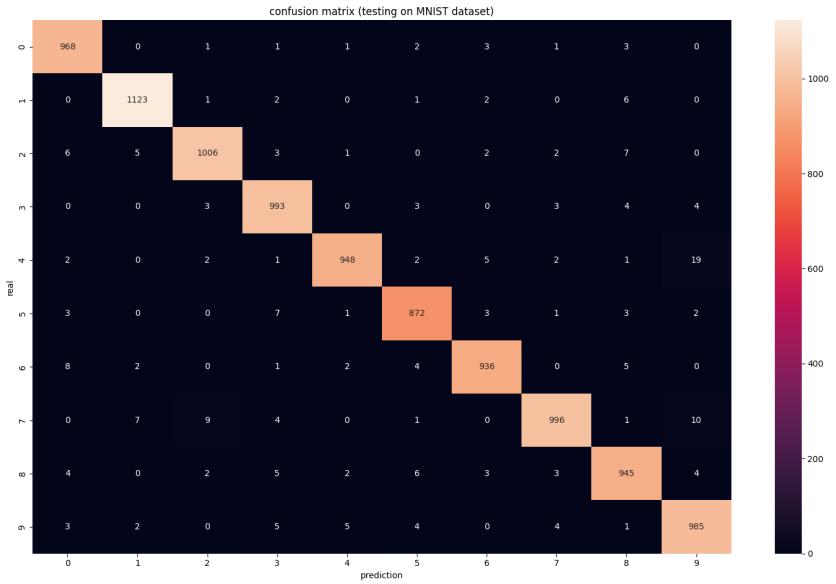


Figure 6: Confusion matrix dla testów na zbiorze EMNIST MNIST

Import własnego zbioru testowego

```
x_my = np.empty([30, 28, 28])
y_my = np.array([i for i in range(num_classes) for _ in
    range(3)])

i = 0
for digit in range (0, num_classes):
        for number in range(1, 4):
                image =
                    Image.open(f"myDigits/{digit}-{number}.png").convert('L')
```

```
                image = image.resize((28, 28))
                x_my[i] = np.array(image)
                i += 1

x_my /= 255.0
x_my = x_my.reshape(x_my.shape[0], -1)

y_my = to_categorical(y_my, num_classes)
```

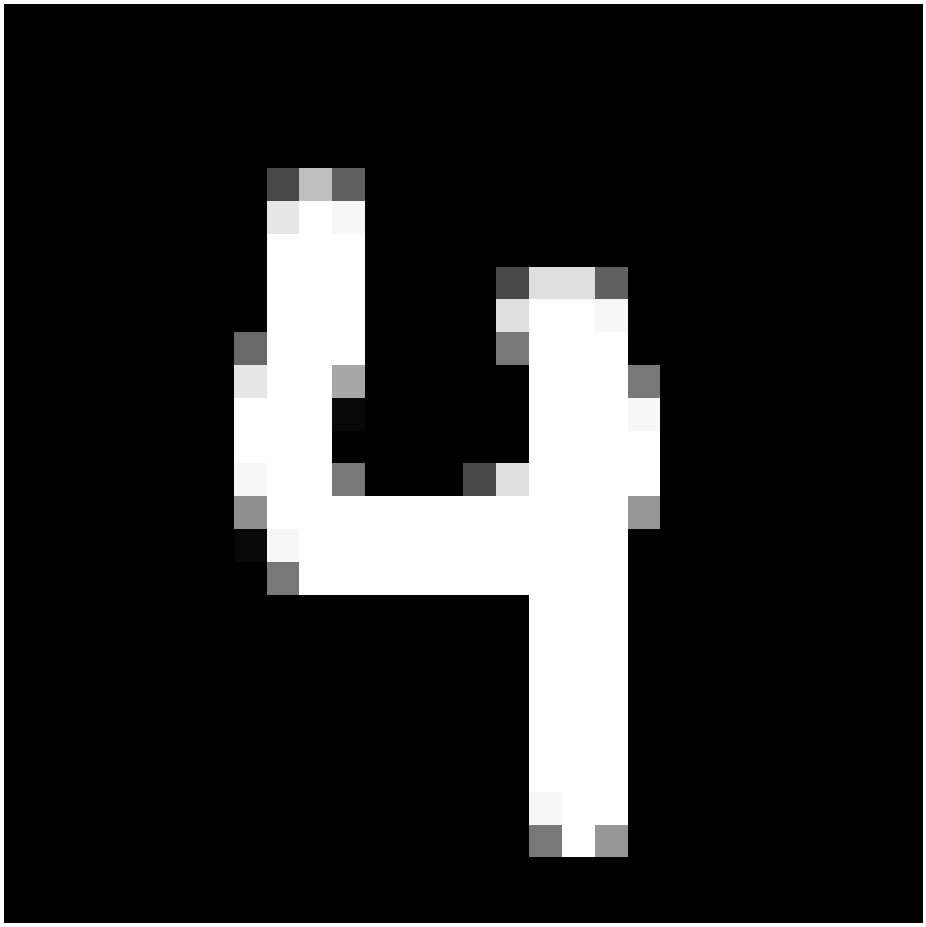


Figure 7: Przykładowa cyfra z mojego zbioru testowego

Testy na własnym zbiorze testowym

```python
# testowanie na w asnym zbiorze
print("testing on my dataset:")
loss, acc, precision, recall = model.evaluate(x_my, y_my)

print(f"accuracy: {acc*100:.2f}%")
print(f"loss: {loss*100:.2f}%")
print(f"precision: {precision*100:.2f}%")
print(f"recall: {recall*100:.2f}%")

y_pred_prob = model.predict(x_my)
y_pred = np.argmax(y_pred_prob, axis=1)
y_real = np.argmax(y_my, axis=1)
conf_matrix = confusion_matrix(y_real, y_pred)

fig, ax = plt.subplots(figsize=(15, 10))
ax = sb.heatmap(conf_matrix, annot=True, fmt='d', ax=ax)
ax.set_xlabel('prediction')
ax.set_ylabel('real')
ax.set_title('confusion matrix (testing on my dataset)')

plt.tight_layout()
plt.show()
```

```
testing on my dataset:
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 135ms/step - acc: 0.7333 - loss: 1.1488 - precision: 0.7333 - recall: 0.7333
accuracy: 73.33%
loss: 114.88%
precision: 73.33%
recall: 73.33%
```

Figure 8: Statystyki: accuracy, precision, recall, loss, w teście na moim zbiorze

Wnioski: Z moim zbiorem testowym sieć poradziła sobie gorzej niż ze zbiorem EMNIST MNIST (73.33% vs 97.72%). Może to być spowodowane różnicą w sposobie pisania cyfr, np. dla cyfry 7 pisanej bez kreski w zbiorze EMNIST MNIST, albo 9 z prostą kreską pionową zamiast zaokrąglonej.

Tworzenie i trenowanie Random Forest

```python
# tworzenie modelu
model = RandomForestClassifier(n_estimators=10,
    random_state=42)

# trenowanie modelu
model.fit(x_train, y_train)
```

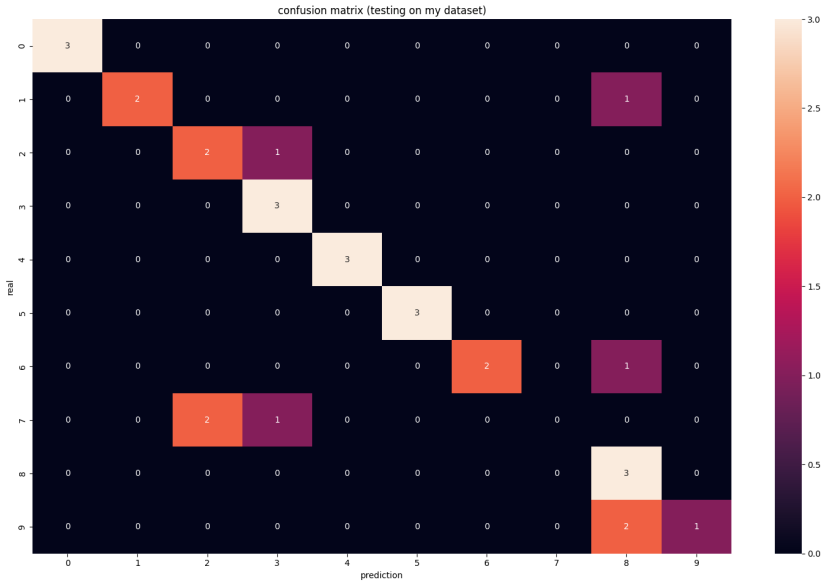


Figure 9: Confusion matrix dla testów na moim zbiorze

Testowanie Random Forest na zbiorze EMNIST MNIST

```
# testowanie na zbiorze MNIST
print("testing on MNIST dataset:")
y_pred_prob = model.predict(x_test)

acc = accuracy_score(y_test, y_pred_prob)
precision = precision_score(y_test, y_pred_prob,
    average='macro')
recall = recall_score(y_test, y_pred_prob,
    average='macro')

print(f"accuracy: {acc*100:.2f}%")
print(f"precision: {precision*100:.2f}%")
print(f"recall: {recall*100:.2f}%")

y_pred = np.argmax(y_pred_prob, axis=1)
y_real = np.argmax(y_test, axis=1)
conf_matrix = confusion_matrix(y_real, y_pred)

fig, ax = plt.subplots(figsize=(15, 10))
ax = sb.heatmap(conf_matrix, annot=True, fmt='d', ax=ax)
```

```
ax.set_xlabel('prediction')
ax.set_ylabel('real')
ax.set_title('confusion matrix (testing on MNIST
    dataset)')

plt.tight_layout()
plt.show()
```

```
testing on MNIST dataset:
accuracy: 86.43%
precision: 99.02%
recall: 86.23%
```

Figure 10: Statystyki: accuracy, precision, recall, loss, w teście Random Tree na zbiorze EMNIST MNIST

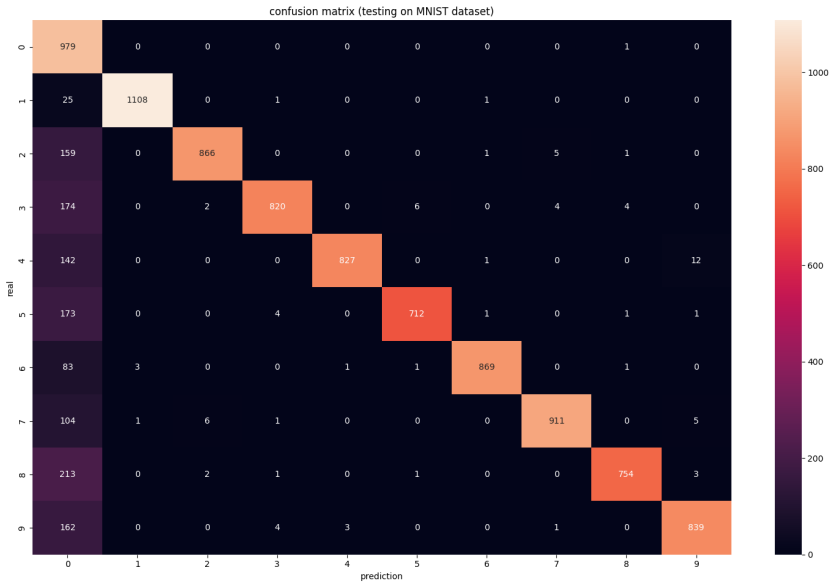


Figure 11: Confusion matrix dla testów Random Tree na zbiorze EMNIST MNIST

Testy Random Tree na własnym zbiorze testowym

```
# testowanie na w asnym zbiorze
print("testing on my dataset:")
y_pred_prob = model.predict(x_my)
```

```
acc = accuracy_score(y_my, y_pred_prob)
precision = precision_score(y_my, y_pred_prob,
    average='macro')
recall = recall_score(y_my, y_pred_prob, average='macro')

print(f"accuracy: {acc*100:.2f}%")
print(f"precision: {precision*100:.2f}%")
print(f"recall: {recall*100:.2f}%")

y_pred = np.argmax(y_pred_prob, axis=1)
y_real = np.argmax(y_my, axis=1)
conf_matrix = confusion_matrix(y_real, y_pred)

fig, ax = plt.subplots(figsize=(15, 10))
ax = sb.heatmap(conf_matrix, annot=True, fmt='d', ax=ax)
ax.set_xlabel('prediction')
ax.set_ylabel('real')
ax.set_title('confusion matrix (testing on my dataset)')

plt.tight_layout()
plt.show()
```

```
accuracy: 33.33%
precision: 53.33%
recall: 33.33%
```

Figure 12: Statystyki: accuracy, precision, recall, loss, w teście Random Tree na moim zbiorze

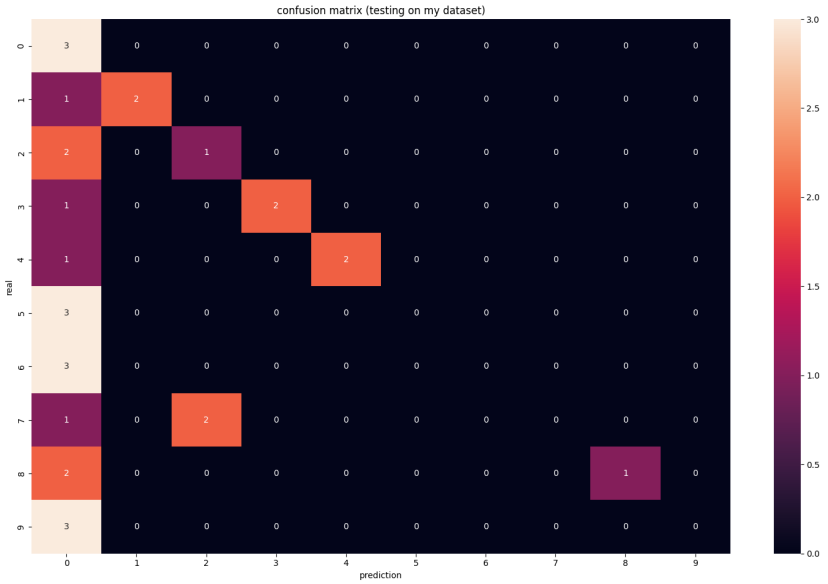


Figure 13: Confusion matrix dla testów Random Tree na moim zbiorze